

รายงานงานปฏิบัติการกลุ่ม

การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack โดยใช้ภาษา Java

กลุ่มที่ 4: Expression Processor (Infix และ Postfix)

ฉบับจัดทำใหม่จากใบงานและรายงานกลุ่มเดิม

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

นิพจน์ทางคณิตศาสตร์ที่มนุษย์เขียนทั่วไปอยู่ในรูป Infix เช่น $3 + 4 * 2$ โดย Operator อยู่ระหว่าง Operand แต่คอมพิวเตอร์ต้องพิจารณา Operator Precedence และ Parentheses ก่อนคำนวณ จึงเหมาะที่จะใช้ Stack ช่วยจัดลำดับการประมวลผล.

1.2 Case Story และวัตถุประสงค์

ระบบ Expression Processor รับนิพจน์ Infix ตรวจสอบความถูกต้อง แปลง Infix เป็น Postfix และคำนวณผลลัพธ์ พร้อมเปรียบเทียบกับผลการประเมิน Infix โดยตรงด้วย Stack สองตัว.

- วิเคราะห์ Input, Output และข้อจำกัด
- ออกแบบ Algorithm A และ Algorithm B สำหรับปัญหาเดียวกัน
- เขียน Pseudocode และ Step-by-step
- อธิบายความถูกต้องด้วย Loop Invariant
- วิเคราะห์ Time/Space Complexity
- พัฒนาโปรแกรม Java พร้อม Test Cases และการทดลองประสิทธิภาพ

1.3 ขอบเขตงาน

- Operator: +, -, *, /
- Parentheses: ()
- Operand: จำนวนเต็มบวกและจำนวนหลายหลัก
- รองรับช่องว่างระหว่าง Token
- ตรวจสอบวงเล็บไม่สมดุล, Operator ผิดตำแหน่ง, Division by Zero และ Empty Expression

- ใช้ Integer Division ตาม Java int / int
- ไม่บังคับรองรับทศนิยม, Unary Minus, ตัวแปร, ฟังก์ชัน และวงเล็บ [] {}

บทที่ 2 การวิเคราะห์ปัญหา

รายการ	รายละเอียด
Input	String ของ Infix เช่น "3 + 4 * 2 / (1 - 5)"
Output	Token list, Validation result, Postfix (Algorithm A), Integer result
Constraints	ไม่รองรับทศนิยม/Unary Minus/ตัวแปร; ใช้เฉพาะ ()
Assumptions	การหารเป็น Integer Division
Edge Cases	Empty, Missing ')', Extra ')', 3 + * 4, Division by zero

2.1 Operation ที่เกิดขึ้นบ่อย

การอ่าน Token และการ push/pop ของ Operand และ Operator เกิดขึ้นตลอดการทำงาน ส่วนการเปรียบเทียบ Precedence อาจเกิดขึ้นในช่วงที่มี Operator ความสำคัญเท่ากันหรือต่ำกว่าบน Stack.

2.2 กรณีปกติและกรณีขอบเขต

ประเภท	ตัวอย่าง	ผลที่คาดหวัง
Normal	3 + 4 * 2	11
Normal with parentheses	(3 + 4) * 2	14
Boundary	(((2 + 3))) * 4	20
Error	(3 + 4	Validation Error
Error	3 + 4)	Validation Error
Error	3 + * 4	Validation Error
Error	10 / (5 - 5)	ArithmeticException
Empty	""	Validation Error

2.3 Operation ที่อาจใช้เวลามากที่สุด

Operation ที่อาจใช้เวลามากที่สุดในการประมวลผลต่อ 1 Token คือ การดึง Operator ออกจาก Stack (Pop) แบบต่อเนื่องภายในลูป (While loop) ซึ่งจะเกิดขึ้นใน 2 สถานการณ์หลัก ได้แก่:

1. เมื่ออ่านพบวงเล็บปิด): โปรแกรมจะต้องทำการ Pop Operator ที่ค้างอยู่ใน Stack ออกมาประมวลผล (หรือนำไปต่อท้าย Postfix) อย่างต่อเนื่องไปเรื่อยๆ จนกว่าจะพบวงเล็บเปิด (
2. เมื่ออ่านพบ Operator ที่มีลำดับความสำคัญ (Precedence) ต่ำกว่าหรือเท่ากับ Operator บนสุดของ Stack: ตัวอย่างเช่น หากใน Stack มี * หรือ / ซ่อนกันอยู่หลายตัว แล้วโปรแกรมอ่านเจอ + หรือ - โปรแกรมจะต้องทำการ Pop Operator ที่มีความสำคัญสูงกว่าหรือเท่ากันออกมาประมวลผลให้หมดเสียก่อน จึงจะสามารถ Push เครื่องหมายใหม่ลงไปได้

เหตุผล: แม้ว่าคำสั่ง Push และ Pop 1 ครั้งจะใช้เวลาคงที่ $O(1)$ แต่ในสถานการณ์เลวร้ายที่สุดของบาง Token (Worst-case per token) เช่น ตอนปิดวงเล็บที่ครอบนิพจน์ที่ยาวมาก หรือการเคลียร์ Operator ทั้งหมดเมื่อจบชุดคำสั่ง อาจทำให้เกิดการทำงานของ While loop หลายรอบติดต่อกันในจังหวะเดียว อย่างไรก็ตาม เมื่อวิเคราะห์ตลอดทั้งการทำงานด้วย Amortized Analysis จะพบว่า Operator แต่ละตัวถูกจำกัดให้เข้าและออกจาก Stack ได้เพียงครั้งเดียว เวลาการทำงานรวมทั้งหมดจึงยังคงเป็น $O(n)$

บทที่ 3 การออกแบบอัลกอริทึม

3.1 Algorithm A: Infix → Postfix → Evaluation

แบ่งเป็น 2 เฟส: ใช้ Operator Stack แปลง Infix เป็น Postfix ตามหลัก Shunting-Yard แล้วใช้ Operand Stack ประเมิน Postfix.

```
ALGORITHM InfixToPostfix(tokens)
```

1. Create empty operatorStack and postfixList
2. FOR EACH token
3. IF number: append to postfixList
4. ELSE IF "(": push to operatorStack
5. ELSE IF ")": pop operators until "("; remove "("
6. ELSE operator: pop operators with precedence $\geq$ current; push current
7. END FOR
8. Pop remaining operators to postfixList
9. RETURN postfixList

```
ALGORITHM EvaluatePostfix(postfixTokens)
```

1. Create empty operandStack
2. FOR EACH token
3. IF number: push integer
4. ELSE: pop val2, pop val1, calculate val1 op val2, push result
5. RETURN the remaining value

Time Complexity: $O(n)$. Space Complexity: $O(n)$. ข้อดีคือ Postfix สามารถเก็บไว้ใช้ซ้ำได้; ข้อจำกัดคือมีการประมวลผล 2 เฟสและมี postfixList เพิ่มเติม.

3.2 Algorithm B: Direct Infix Evaluation

ประมวลผล Infix ในรอบเดียว โดยใช้ Operand Stack เก็บค่า และ Operator Stack เก็บเครื่องหมาย เมื่อพบ Operator ใหม่จะประมวลผล Operator ที่มี Precedence สูงกว่าหรือเท่ากันก่อน.

```
ALGORITHM EvaluateInfixDirect(tokens)
```

1. Create empty operandStack and operatorStack
2. FOR EACH token
3. IF number: push to operandStack
4. ELSE IF "(": push to operatorStack
5. ELSE IF ")": apply operators until "("; remove "("
6. ELSE operator: apply operators with precedence $\geq$ current; push current
7. END FOR
8. Apply all remaining operators
9. RETURN operandStack.pop()

```
PROCEDURE ApplyTopOperator
```

1. op $\leftarrow$ operatorStack.pop()
2. val2 $\leftarrow$ operandStack.pop()
3. val1 $\leftarrow$ operandStack.pop()
4. push ExecuteOperation(val1, val2, op)

Time Complexity: $O(n)$. Space Complexity: $O(n)$. ข้อดีคือ Single Pass และไม่ต้องเก็บ Postfix; ข้อจำกัดคือ Logic รวมกันในรูปแบบเดียวและไม่สามารถนำ Postfix ไปใช้ซ้ำได้โดยตรง.

3.3 ตัวอย่างบังคับ

Infix: $3 + 4 * 2 / (1 - 5)$

Postfix: $3\ 4\ 2\ *\ 1\ 5\ -\ /\ +$

Result: 1

Step	Token	Operator Stack	Postfix / Action
1	3	[]	3
2	+	[+]	3
3	4	[+]	3, 4
4	*	[+, *]	3, 4
5	2	[+, *]	3, 4, 2
6	/	[+, /]	Pop * $\rightarrow$ 3, 4, 2, *
7	(	[+, /, (]	unchanged
8	1	[+, /, (]	..., 1
9	-	[+, /, (, -]	..., 1
10	5	[+, /, (, -]	..., 1, 5
11	)	[+, /]	Pop -
12	End	[]	Pop /, + $\rightarrow$ final Postfix

สำหรับ Algorithm B เมื่ออ่านนิพจน์เดียวกัน ผลลัพธ์สุดท้ายคือ 1 เช่นเดียวกับ Algorithm A.

3.4 ตัวอย่างบังคับกรณีขอบเขต

Infix: $((2 + 3)) * 4$

Postfix: $2\ 3\ +\ 4\ *$

Result: 20

Step	Token	Operator Stack	Postfix / Action
1	(	[(	unchanged
2	(	[(, (	unchanged
3	(	[(, (, (	unchanged
4	2	[(, (, (	2
5	+	[(, (, (, +]	2
6	3	[(, (, (, +]	2, 3
7	)	[(, (	Pop + → 2, 3, +
8	)	[(	Pop (→ unchanged
9	)	[	Pop (→ unchanged
10	*	[*]	2, 3, +
11	4	[*]	2, 3, +, 4
12	End	[	Pop * → final: 2, 3, +, 4, *

3.5 Flowcharts

Figure 3A — Algorithm A Flowchart

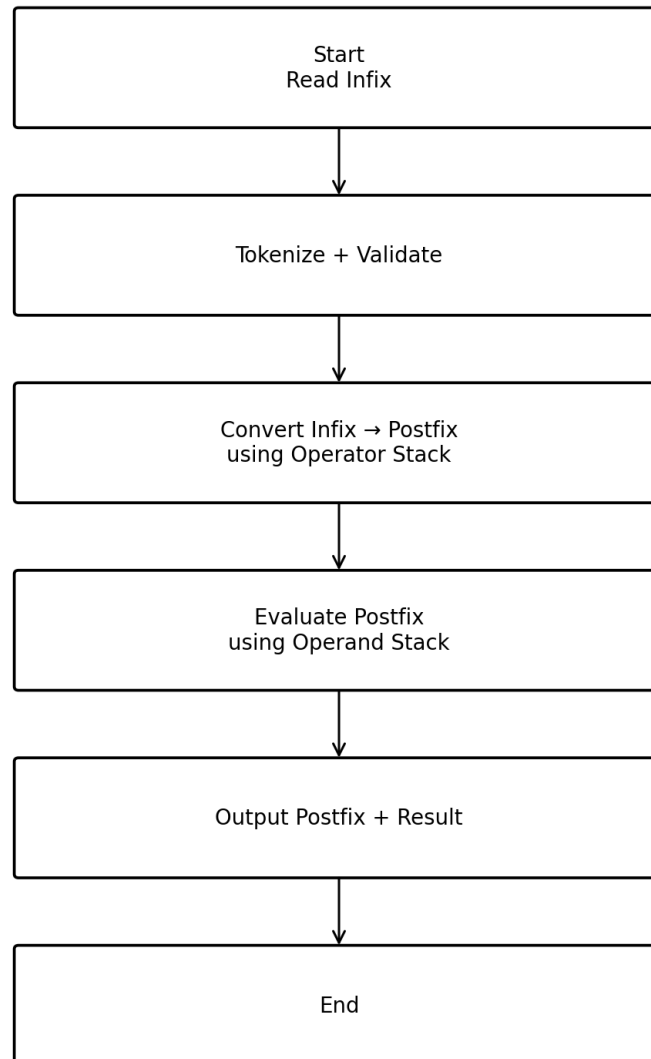
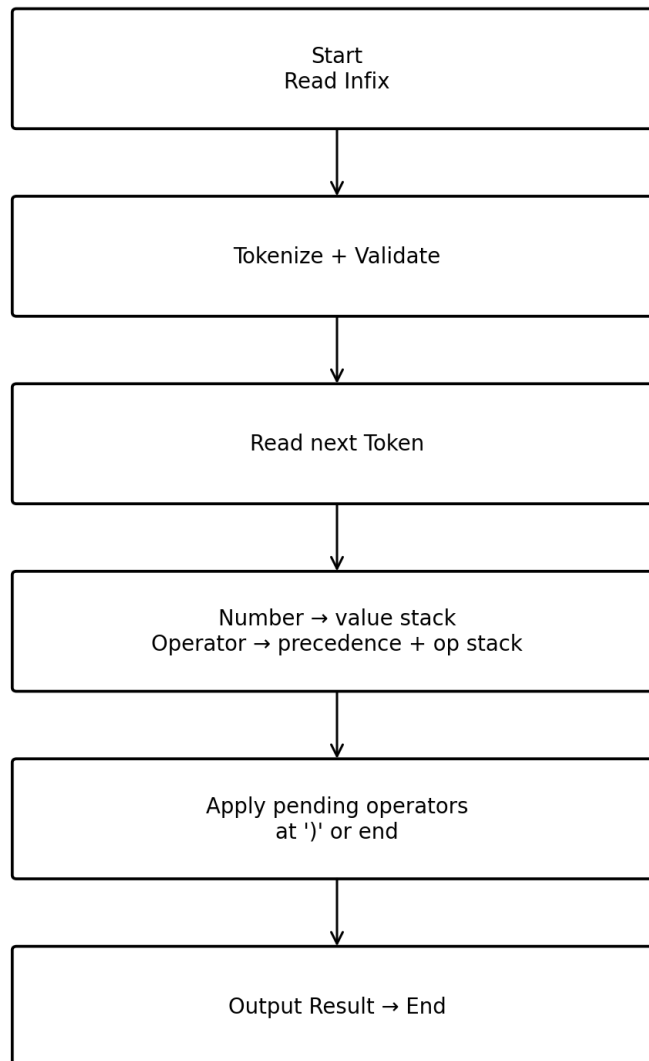


Figure 3B — Algorithm B Flowchart



บทที่ 4 การอธิบายความถูกต้อง

4.1 Precondition

Input เป็นนิพจน์ Infix ที่ผ่านการ Tokenize และ Validation แล้ว ไม่มีวงเล็บไม่สมดุล ไม่มี Operator ผิดตำแหน่ง และไม่มี Empty Expression.

4.2 Loop Invariant ของ Algorithm B

ก่อนเริ่มรอบที่ประมวลผล Token ไปแล้ว i ตัว: Operand Stack เก็บผลลัพธ์ของนิพจน์ย่อยที่คำนวณเสร็จจากซ้ายไปขวาโดยไม่สูญหาย และ Operator Stack เก็บเฉพาะ Operator/วงเล็บเปิดที่ยังรอการประมวลผล โดยลำดับ Precedence ภายในขอบเขตวงเล็บไม่ลดลงจากล่างขึ้นบน.

4.3 Initialization

เมื่อ $i = 0$ Stack ทั้งสองว่าง จึงเป็นจริงโดยปริยาย.

4.4 Maintenance

- Number: push เป็นนิพจน์ย่อยที่สมบูรณ์
- (: สร้าง Boundary ใหม่ใน Operator Stack
- Operator: ประมวลผล Operator ที่มี Precedence สูงกว่าหรือเท่ากันก่อน แล้วจึง push ตัวใหม่
-): ประมวลผลจนถึง '(' และลบ Boundary นั้น

4.5 Termination และ Postcondition

เมื่ออ่าน Token ครบ Operator Stack จะถูกประมวลผลจนว่าง และ Operand Stack เหลือค่าหนึ่งค่า ซึ่งเป็นผลลัพธ์ของนิพจน์ทั้งหมด. การวนซ้ำสิ้นสุดเพราะจำนวน Token มีจำกัด และแต่ละ while iteration ลดจำนวน Operator ใน Stack.

4.6 การรักษาลำดับข้อมูล

การคำนวณแบบ **non-commutative** รักษาลำดับซ้าย-ขวาโดยกำหนด **val2** เป็นค่าที่ **pop** ครั้งแรกและ **val1** เป็นค่าที่ **pop** ครั้งที่สอง แล้วคำนวณ **val1 op val2**.

4.7 การอธิบายความถูกต้องตามข้อกำหนดเฉพาะของกลุ่มที่ 4

เพื่อให้การทำงานของอัลกอริทึมมีความถูกต้องและรักษาความหมายทางคณิตศาสตร์ไว้ได้อย่างสมบูรณ์ สามารถอธิบายกลไกการทำงานได้ดังนี้

1. Operator ที่มี Priority สูงกว่าถูกประมวลผลก่อนอย่างไร ระบบใช้การเปรียบเทียบลำดับความสำคัญ (Precedence) ก่อนนำ Operator ตัวใหม่ลง Stack เสมอ หาก Operator ตัวใหม่มีความสำคัญ "ต่ำกว่าหรือเท่ากับ" ตัวที่อยู่บนสุดของ Stack ระบบจะดึง (Pop) ตัวที่อยู่บนสุดออกมาประมวลผลก่อน แล้วจึงนำตัวใหม่ใส่ลงไป กลไกนี้รับประกันว่าเครื่องหมายที่มีความสำคัญสูงกว่า (เช่น $*$ หรือ $/$) ที่เข้ามาก่อน จะถูกดึงไปประมวลผลก่อนเครื่องหมายที่มีความสำคัญต่ำกว่า (เช่น $+$ หรือ $-$) ที่ตามมาทีหลังเสมอ

2. วงเล็บเปลี่ยนลำดับการคำนวณอย่างไร วงเล็บเปิด (ทำหน้าที่เป็นขอบเขตหรือกำแพงกั้นใน Operator Stack เมื่อมี (เข้าไปใน Stack Operator ตัวอื่นๆ ที่ตามมาจะถูกวางทับลงไปโดยไม่สามารถเปรียบเทียบหรือประมวลผลร่วมกับ Operator ที่อยู่ได้ (ได้จนกว่าระบบจะอ่านพบวงเล็บปิด) ระบบจึงจะบังคับ Pop และประมวลผล Operator ทุกตัวที่อยู่เหนือ (ออกมาจนหมด กลไกนี้เป็นการบังคับให้ระบบคำนวณนิพจน์ย่อยภายในวงเล็บให้เสร็จสิ้นก่อนเสมอ จึงจะสามารถกลับไปคำนวณนิพจน์ภายนอกวงเล็บต่อไปได้

3. Postfix รักษาความหมายของ Infix อย่างไร นิพจน์แบบ Infix ต้องใช้กฎลำดับความสำคัญและวงเล็บเพื่อระบุลำดับการทำงาน แต่นิพจน์แบบ Postfix จะจัดเรียงลำดับเหล่านั้นไว้ในตำแหน่งของข้อมูลอย่างชัดเจน โดยวาง Operator ไว้หลัง Operand เสมอ ในขั้นตอนการแปลงแบบ Algorithm A อัลกอริทึมได้ใช้ Stack จัดการกับลำดับความสำคัญและถอดวงเล็บออกไปทั้งหมดแล้ว ดังนั้น การอ่านนิพจน์ Postfix จากซ้ายไปขวาจึงเป็นลำดับการคำนวณที่แท้จริงแบบตรงไปตรงมา การประเมินค่าสามารถทำได้ทันทีโดยไม่ต้องพิจารณาวงเล็บซ้ำ ทำให้ผลลัพธ์สุดท้ายถูกต้องตรงตามความหมายของ Infix ต้นฉบับ

4. Operand Stack เก็บผลลัพธ์ย่อยอย่างไร ในระหว่างการประมวลผล เมื่อมีการดึง Operator 1 ตัว มาใช้งาน ระบบจะดึง (Pop) ตัวเลข 2 ค่าจาก Operand Stack มาคำนวณหาผลลัพธ์ (เช่น ดึง 3 กับ 4 มาบวกกันได้ 7) จากนั้นระบบจะนำผลลัพธ์ที่ได้ (7) ใส่ (Push) กลับลงไปใน Operand Stack ทันที ค่าที่ถูกใส่กลับลงไปในที่นี้ทำหน้าที่เป็นตัวแทนของนิพจน์ย่อยที่คำนวณเสร็จแล้ว และพร้อมที่จะเป็น Operand สำหรับการคำนวณกับ Operator ตัวถัดไป กระบวนการนี้จะยุบรวมผลลัพธ์ย่อยไปเรื่อยๆ จนกว่าจะเหลือเพียงค่าเดียวใน Stack ซึ่งก็คือคำตอบสุดท้าย

บทที่ 5 การวิเคราะห์ประสิทธิภาพ

Algorithm	Best	Average	Worst	Auxiliary Space
A: Infix → Postfix → Eval	$O(n)$	$O(n)$	$O(n)$	$O(n)$
B: Direct Infix	$O(n)$	$O(n)$	$O(n)$	$O(n)$

แม้จะมี while อยู่ภายใน for แต่การ pop ของแต่ละ Token มีจำนวนจำกัดตลอดการทำงาน จึงใช้ Amortized Analysis และได้ $O(n)$ ไม่ใช่ $O(n^2)$.

5.1 Space

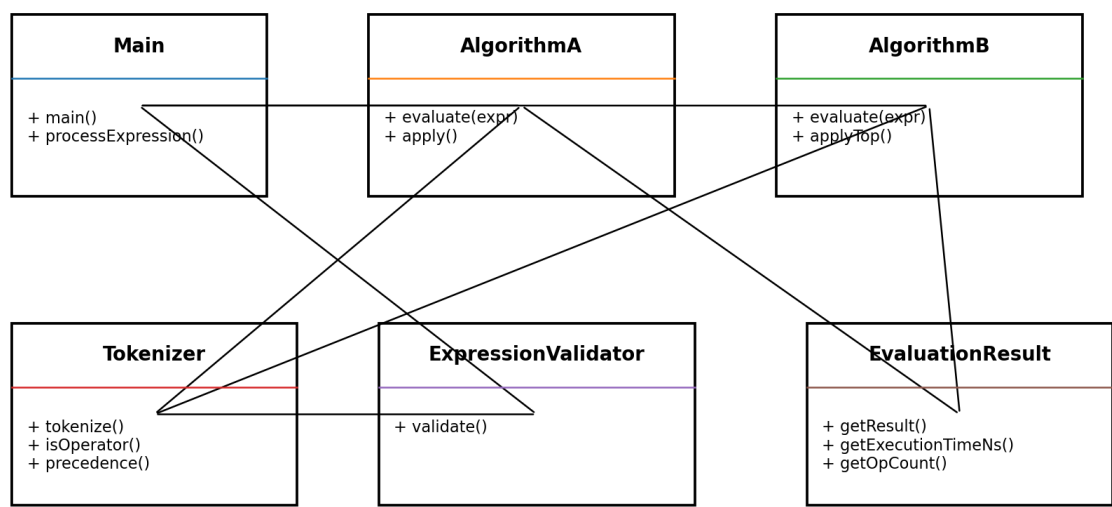
Algorithm A ใช้ Operator Stack + Postfix List + Operand Stack รวม $O(n)$.

Algorithm B ใช้ Operand Stack + Operator Stack รวม $O(n)$ เช่นกัน แต่ไม่มี Postfix List จึงมีพื้นที่ใช้งานจริงน้อยกว่าในบริบทนี้.

บทที่ 6 การพัฒนาโปรแกรม Java

Class	หน้าที่
group4.Main	เมนู รับ Input เรียก A/B และรัน Test Cases
group4.models.EvaluationResult	เก็บ Result, Postfix, Time และ Operation Counts
group4.utils.Tokenizer	แปลง String เป็น Token และกำหนด Precedence
group4.utils.ExpressionValidator	ตรวจ Empty, Parentheses และ Token Sequence
group4.algorithms.AlgorithmA	Infix → Postfix → Evaluation
group4.algorithms.AlgorithmB	Direct Infix ด้วย 2 Stacks
group4.ExpressionGenerator	สร้างนิพจน์สำหรับ Performance Test
group4.ExperimentRunner	รัน n = 100, 1,000, 10,000, 50,000 อย่างละ 5 รอบ

Figure 6.1 — Simplified Class Diagram



6.1 Input Validation และ Error Handling

- Invalid character → `IllegalArgumentException`
- Unbalanced parentheses → `Validation Error`
- Operator sequence ผิด → `Validation Error`
- Empty expression → `Validation Error`
- Division by zero → `ArithmeticException` / `Execution Error`
- โปรแกรมไม่ Crash และกลับสู่เมนูหลัก

6.2 การแสดงสถานะของ Stack (Step-by-step Trace)

เพื่อให้สอดคล้องกับข้อกำหนดของโจทย์ โปรแกรมได้เพิ่มฟังก์ชันสำหรับแสดงสถานะของ Stack แบบทีละขั้นตอน (Step-by-step Trace) ไว้ในตัวเลือกเมนูหมายเลข 2 ของ Main Class เมื่อผู้ใช้เลือกตัวเลือกนี้และป้อนนิพจน์ ระบบจะเรียกใช้เมธอด `printStackState` ภายใน Algorithm B เพื่อพิมพ์สถานะปัจจุบันของ Token ที่กำลังอ่าน, ข้อมูลภายใน Operator Stack, และข้อมูลภายใน Operand Stack ออกมาแสดงผลทางหน้าจอแบบบรรทัดต่อบรรทัด ฟังก์ชันนี้ช่วยให้สามารถตรวจสอบการไหลของข้อมูล (Data Flow) ได้อย่างชัดเจน และใช้สำหรับการสาธิตการทำงานของอัลกอริทึมระหว่างการทำงานนำเสนอ

```
private static void printStackState(String currentToken, Deque<String> ops, Deque<Integer> values)
{
    // สร้าง Array จากล่างขึ้นบนเพื่อให้แสดงผลดูง่าย
    Object[] opsArray = ops.toArray();
    Object[] valuesArray = values.toArray();

    StringBuilder opsStr = new StringBuilder("[");
    for (int i = opsArray.length - 1; i >= 0; i--) {
        opsStr.append(opsArray[i]).append(i > 0 ? ", " : "");
    }
    opsStr.append("]");

    StringBuilder valStr = new StringBuilder("[");
    for (int i = valuesArray.length - 1; i >= 0; i--) {
        valStr.append(valuesArray[i]).append(i > 0 ? ", " : "");
    }
    valStr.append("]");

    System.out.printf("%-5s | %-14s | %s\n", currentToken, opsStr.toString(), valStr.toString());
}
```

บทที่ 7 ผลการทดสอบและการทดลองประสิทธิภาพ

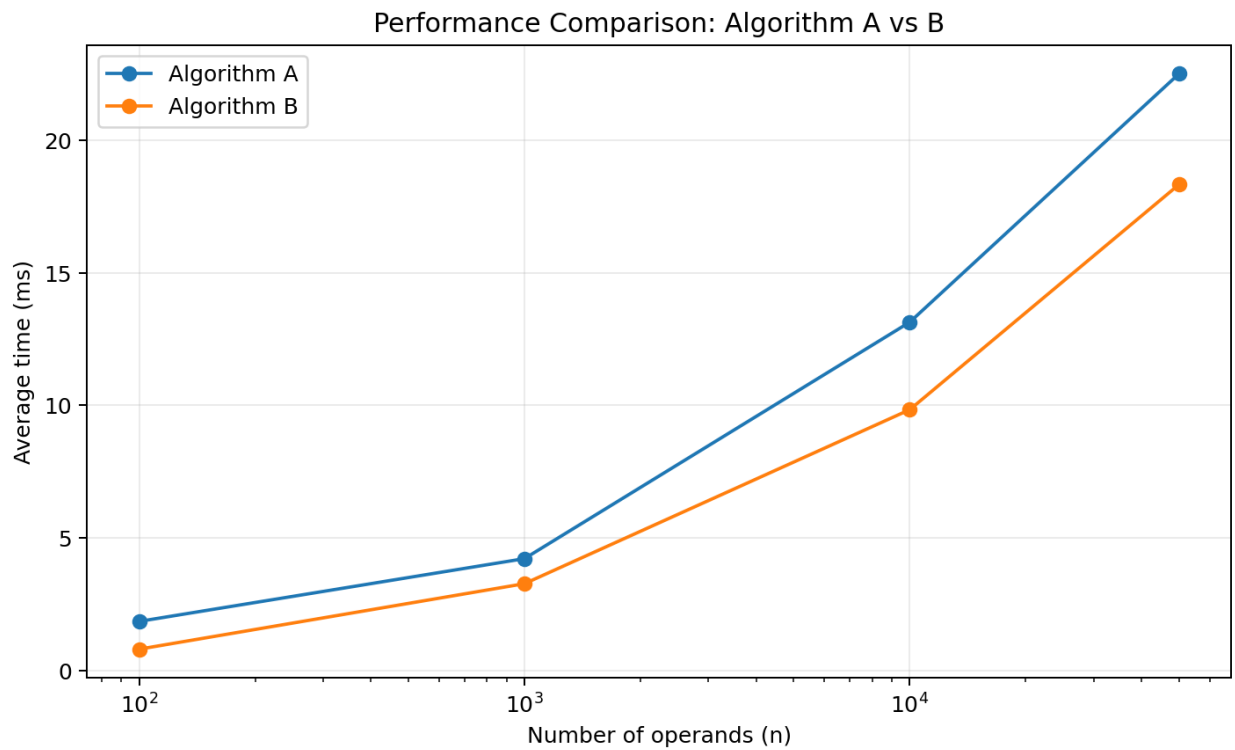
7.1 Mandatory Test Cases

ID	Input	Expected	Actual	Status
TC01	$3 + 4 * 2$	Result = 11	Result = 11	PASSED
TC02	$(3 + 4) * 2$	Result = 14	Result = 14	PASSED
TC03	$((8 + 2) * 5)$	Result = 50	Result = 50	PASSED
TC04	$(3 + 4$	Error: Missing ')' at end	Validation Error Caught	PASSED
TC05	$3 + 4)$	Error: Extra ')' found	Validation Error Caught	PASSED
TC06	$3 + * 4$	Error: Invalid operator sequence	Validation Error Caught	PASSED
TC07	$10 / (5 - 5)$	Error: Division by zero	ArithmeticException Caught	PASSED
TC08		Error: Expression is empty	Validation Error Caught	PASSED

7.2 Performance Experiment

ทดลอง $n = 100, 1,000, 10,000$ และ $50,000$ operands ขนาดละ 5 รอบ แล้วใช้ค่าเฉลี่ยของ Execution Time, Push, Pop และ Comparisons ตามข้อกำหนดในใบงาน.

n	Algorithm	Avg Time (ns)	Push	Pop	Comparisons
100	A	1,852,512	298.0	297.0	296.4
100	B	805,085	298.0	297.0	296.4
1000	A	4,219,254	2998.0	2997.0	2996.4
1000	B	3,274,884	2998.0	2997.0	2996.4
10000	A	13,138,604	29998.0	29997.0	29996.6
10000	B	9,837,999	29998.0	29997.0	29996.6
50000	A	22,509,382	149998.0	149997.0	149996.2
50000	B	18,332,269	149998.0	149997.0	149996.2



ผลการทดลองจากรายงานกลุ่มเดิมแสดงว่า **Algorithm B** เร็วกว่า **Algorithm A** ทุกขนาดที่ทดสอบ แม้ทั้งสองมี $O(n)$. ตัวอย่าง $n = 50,000$: A ประมาณ 22.5 ms และ B ประมาณ 18.3 ms.

7.3 อภิปรายผล

จำนวน **Push/Pop** เพิ่มขึ้นตามขนาดข้อมูลอย่างเป็นเชิงเส้น ซึ่งสอดคล้องกับ $O(n)$. ความแตกต่างด้านเวลามาจาก **Constant Factor**: Algorithm A มี 2 เฟสและต้องสร้าง **Postfix List** ขณะที่ Algorithm B ประมวลผลแบบ **Single Pass**.

บทที่ 8 บทปฏิบัติการใช้ Generative AI

งาน	Prompt ที่ใช้	คำตอบหรือ Source Code ที่ AI สร้าง	ผลลัพธ์จาก AI (สรุป)	วิธีตรวจสอบ	สิ่งที่กลุ่มแก้ไข
Tokenization	"เขียนโค้ดแยก Token สำหรับ สมการคณิตศาสตร์ใน Java ให้หน่อย"	<code>`String[] tokens = expr.split("(?<=[-+*/()])");`</code>	<code>(?=[-+*/()])");`</code>	เสนอให้ใช้ regex split	ทดสอบกับ $(3+4)*2$ และตัวเลขหลัก พบว่าทำงานผิดพลาดกับช่องว่าง
Big-O	"Algorithm ที่มี while loop ซ่อนอยู่ใน for loop ในการผลักค่าลง Stack มี Time Complexity เป็นเท่าใด"	"มี Time Complexity เป็น $O(n^2)$ เนื่องจากมีลูป ซ่อนกันสองชั้น..."	เสนอ $O(n^2)$ จาก loop ซ่อน	ทำการ Trace Stack ด้วยมือ และ ใช้หลักการ Amortized Analysis	แก้ไขข้อมูลในรายงาน เป็น $O(n)$ พร้อมเขียนอธิบายเหตุผลประกอบ
Test Cases	"ช่วยคิด Test Cases สำหรับ โปรแกรมคำนวณ Infix เป็น Postfix หน่อย"	เสนอเคสพื้นฐาน 5 เคส เช่น $3+5$, $10*20$, $(1+2)*3$	เสนอกรณีทดสอบทั่วไป	นำไปเทียบกับ 8 Mandatory Cases ที่กำหนดไว้ในใบงาน	เพิ่มกรณี Boundary Cases และ Error Cases (เช่น ขาดวงเล็บ, หารศูนย์) ให้ครบถ้วน

Experiment / Graph	"ขอโค้ด Java สำหรับจับเวลาและนับ Operation ในการรันฟังก์ชัน"	แนะนำการใช้ System.nanoTime() และการบวกตัวแปรนับจำนวนในโค้ด	ช่วยร่างเครื่องมือและโครงสร้างการวัดผล	นำไป Compile/Run และเทียบ operation counts กับการจำลองด้วยมือ	แยก Helper Class ออกมาสำหรับการทดลองโดยเฉพาะ เพื่อไม่ให้รบกวน Logic ของ Algorithm A/B
--------------------	--	---	--	---	---

Reflection: AI ใช้เพื่อช่วยเรียนรู้และร่างแนวทางเท่านั้น ผลลัพธ์ทุกส่วนต้องถูกตรวจสอบและผู้จัดทำต้องอธิบาย Source Code ได้.

บทที่ 9 สรุปผลและการเปรียบเทียบ

9.1 ผลการเปรียบเทียบ

ประเด็น	Algorithm A	Algorithm B
แนวทาง	Infix $\rightarrow$ Postfix $\rightarrow$ Evaluate	Direct Infix
Time	$O(n)$	$O(n)$
Space	$O(n)$	$O(n)$
Postfix ใช้ซ้ำ	ได้	ไม่ได้โดยตรง
One-shot execution	ช้ากว่าในผลทดลอง	เร็วกว่าในผลทดลอง

9.2 วิธีที่เหมาะสมกว่า

สำหรับการคำนวณผลลัพธ์เพียงครั้งเดียว **Algorithm B** เหมาะกว่า เพราะผลทดลองแสดงเวลาต่ำกว่าและไม่ต้องสร้าง **Postfix List**. หากต้องเก็บ **Postfix** เพื่อใช้ซ้ำหรือส่งต่อให้ระบบอื่น **Algorithm A** มีประโยชน์มากกว่า.

9.3 ข้อจำกัดและแนวทางพัฒนา

- ยังไม่รองรับทศนิยม, **Unary Minus**, ตัวแปร และฟังก์ชันทางคณิตศาสตร์
- สามารถพัฒนาต่อด้วย **%**, **^** และ **Associativity** ของ **^**
- เพิ่ม **[]** และ **{}**
- เพิ่มการแสดงผล **Trace** ของ **Stack** แบบละเอียดในโปรแกรม

9.4 สิ่งที่ได้เรียนรู้

เห็นความแตกต่างระหว่าง **Asymptotic Complexity** กับประสิทธิภาพจริงจาก **Constant Factor** และได้ฝึก **Stack**, **Amortized Analysis**, **Loop Invariant**, **Testing** และ **Performance Measurement**.

9.5 การตอบคำถามวิเคราะห์ประจำกลุ่มที่ 4

1. Balanced Parentheses มี Complexity เท่าใด

Time Complexity คือ $O(n)$ และ Space Complexity คือ $O(n)$ เมื่อ n คือ จำนวน Token ในนิพจน์ เนื่องจากระบบทำการอ่าน Token แต่ละตัวเพียง 1 รอบจากซ้ายไปขวา การ Push ลง Stack (เมื่อพบวงเล็บเปิด) และการ Pop (เมื่อพบวงเล็บปิด) ใช้เวลาคงที่ $O(1)$ ต่อการกระทำ 1 ครั้ง ส่วนพื้นที่ใช้สอยในกรณีเลวร้ายที่สุด (เช่น มีแต่วงเล็บเปิดทั้งหมด) จะแปรผันตรงกับ n

2. Infix to Postfix มี Complexity เท่าใด

Time Complexity คือ $O(n)$ และ Space Complexity คือ $O(n)$ แม้ว่าในโค้ดจะมี While loop ซ่อนอยู่ใน For loop เพื่อตรวจสอบลำดับความสำคัญ (Precedence) แต่ Operator แต่ละตัวจะถูกนำเข้าสู่ Stack เพียง 1 ครั้ง และดึงออกเพียง 1 ครั้งตลอดทั้งกระบวนการ ทำให้เวลาการทำงานรวมไม่ได้ทวีคูณเป็นกำลังสอง แต่จะแปรผันตรงกับจำนวน Token ทั้งหมดแบบเชิงเส้น

3. เพราะเหตุใดแต่ละ Token จึงถูก Push และ Pop จำนวนจำกัด

เพราะกลไกการทำงานของ Stack ในอัลกอริทึมนี้กำหนดให้ Token แต่ละตัว (ทั้งตัวเลขและเครื่องหมาย) ถูกอ่านและนำเข้าสู่ (Push) เพียงแค่ 1 ครั้ง และเมื่อมันถูกดึงออก (Pop) เพื่อนำไปใช้คำนวณหรือต่อท้ายนิพจน์แล้ว มันจะไม่ถูกนำกลับเข้าไปใน Operator Stack อีก วงจรชีวิตของ Token แต่ละตัวใน Stack จึงมีแค่การเข้า 1 ครั้งและออก 1 ครั้ง ทำให้จำนวน Operation รวมถูกจำกัดอย่างชัดเจน

4. Algorithm A และ B มีข้อดีและข้อเสียอย่างไร

- **Algorithm A (แปลงเป็น Postfix ก่อนคำนวณ):**

- **ข้อดี:** นิพจน์ Postfix ที่แปลงเสร็จสามารถนำไปบันทึกและเรียกใช้งานซ้ำได้ทันทีโดยไม่ต้องแปลงใหม่ โค้ดมีการแยกความรับผิดชอบชัดเจน (แปลงโครงสร้าง และ คำนวณผลลัพธ์)
- **ข้อเสีย:** ทำงานช้ากว่าในเชิงปฏิบัติเพราะต้องประมวลผล 2 เฟส (วนลูป 2 รอบ) และใช้พื้นที่หน่วยความจำเพิ่มเติมเพื่อเก็บ List ของ Postfix

- **Algorithm B (คำนวณ Infix โดยตรง):**

- **ข้อดี:** ทำงานเร็วกว่าเนื่องจากประมวลผลรวดเดียวจบ (Single Pass) และประหยัดพื้นที่มากกว่าเพราะไม่ต้องจองหน่วยความจำเพื่อเก็บโครงสร้าง Postfix
- **ข้อเสีย:** โค้ดซับซ้อนกว่าเพราะต้องรวมตรรกะการจัดการลำดับความสำคัญและการคำนวณทางคณิตศาสตร์ไว้ในลูปเดียวกัน ไม่สามารถส่งต่อโครงสร้างนิพจน์ที่จัดเรียงแล้วไปให้ระบบอื่นใช้ซ้ำได้

5. วิธีใดตรวจสอบ Error ได้ง่ายกว่า

Algorithm A ตรวจสอบ Error ได้ง่ายกว่าและเป็นระบบมากกว่า เนื่องจากการทำงานถูกแบ่งเป็น 2 เฟสอย่างชัดเจน การตรวจสอบข้อผิดพลาดด้านไวยากรณ์ (เช่น ขาดวงเล็บ หรือวางตำแหน่ง Operator ผิด) สามารถดักจับได้จนจบตั้งแต่เฟสการแปลงเป็น Postfix หากผ่านเฟสแรกมาได้ เฟสที่สองจะโฟกัสแค่การดักจับข้อผิดพลาดทางคณิตศาสตร์ (เช่น การหารด้วยศูนย์) ทำให้

ระบุสาเหตุและตำแหน่งของ Error ได้แม่นยำกว่า Algorithm B ที่ต้องจัดการ Error ทุกประเภทพร้อมกันในรูปเดียว

ภาคผนวก A: โครงสร้าง Repository สำหรับ GitHub

```
stack-algorithm-group04/
├─ src/
│   ├─ Main.java
│   ├─ ExpressionGenerator.java
│   ├─ ExperimentRunner.java
│   ├─ models/
│   ├─ algorithms/
│   └─ utils/
├─ test/
│   └─ test_cases.csv
├─ data/
├─ results/
│   └─ experiment_results.csv
├─ diagrams/
│   ├─ algorithmA_flowchart.png
│   ├─ algorithmB_flowchart.png
│   ├─ class_diagram.png
│   └─ performance_comparison.png
├─ presentation/
├─ report/
├─ README.md
├─ group-members.md
└─ AI_Usage_Log.md
```

ตามใบงาน สมาชิกทุกคนต้องมี **Commit** ใน **Repository** และต้องสามารถอธิบาย **Source Code** ของกลุ่มได้. ตัวอย่าง **Commit**: Implement expression processor; Add mandatory test cases; Add performance experiment; Update complexity analysis.

ภาคผนวก B: Source Code

src/Main.java

```
package group4;

import group4.algorithms.AlgorithmA;
import group4.algorithms.AlgorithmB;
import group4.models.EvaluationResult;
import group4.utils.ExpressionValidator;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.println("=====");
        System.out.println(" Group 4: Expression Processor (Infix & Postfix)");
        System.out.println("=====");

        while (true) {

            System.out.println("\nSelect Menu:");

            System.out.println("1. Test Custom Expression");
            System.out.println("2. Test Custom Expression with Step-by-step Trace");
            System.out.println("3. Run Mandatory Assignment Test Cases (1-8)");
            System.out.println("4. Exit");

            System.out.print("Enter choice (1-4): ");

            String choice = scanner.nextLine().trim();

            if (choice.equals("4")) break;

            if (choice.equals("1")) {
```

```

        System.out.print("Enter Infix Expression: ");

        processExpression(scanner.nextLine(), false);

    } else if (choice.equals("2")) {

        System.out.print("Enter Infix Expression: ");

        processExpression(scanner.nextLine(), true);

    } else if (choice.equals("3")) {

        runMandatoryTestCases();

    } else {

        System.out.println("Invalid choice. Please enter 1, 2, 3, or 4.");

    }

}

scanner.close();

}

private static void processExpression(String expr, boolean showTrace) {

    System.out.println("\n--- Processing: \"" + expr + "\" ---");

    try {

        ExpressionValidator.validate(expr);

        EvaluationResult a = AlgorithmA.evaluate(expr);

        EvaluationResult b = AlgorithmB.evaluate(expr, showTrace);

        System.out.println("\n[Algorithm A - Infix to Postfix]");

        System.out.println("Postfix : " + a.getPostfixExpression());

        System.out.println("Result  : " + a.getResult());

        System.out.println("Time    : " + a.getExecutionTimeNs() + " ns");

        System.out.println("Ops     : " + a.getOpCount());

        System.out.println("\n[Algorithm B - Direct Infix]");

        System.out.println("Result  : " + b.getResult());

        System.out.println("Time    : " + b.getExecutionTimeNs() + " ns");

    } catch (Exception e) {

        System.out.println("Error: " + e.getMessage());

    }

}

```

```

        System.out.println("Ops      : " + b.getOpCount());

    } catch (IllegalArgumentException e) {

        System.out.println("Validation Error: " + e.getMessage());

    } catch (ArithmeticException e) {

        System.out.println("Execution Error: " + e.getMessage());

    }

}

private static void runMandatoryTestCases() {

    String[] testCases = {

        "3 + 4 * 2",

        "(3 + 4) * 2",

        "((8 + 2) * 5)",

        "(3 + 4",

        "3 + 4)",

        "3 + * 4",

        "10 / (5 - 5)",

        ""

    };

    for (int i = 0; i < testCases.length; i++) {

        System.out.println("\n=== Mandatory Test Case " + (i + 1) + " ===");

        processExpression(testCases[i], false);

    }

}

}

```

src/utis/Tokenizer.java

```
package group4.algorithms;

import group4.models.EvaluationResult;
import group4.utis.Tokenizer;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

public class AlgorithmB {

    public static EvaluationResult evaluate(String expr) {
        return evaluate(expr, false);
    }

    public static EvaluationResult evaluate(String expr, boolean showTrace) {
        long start = System.nanoTime();

        int push = 0, pop = 0, comp = 0;

        List<String> tokens = Tokenizer.tokenize(expr);

        Deque<Integer> values = new ArrayDeque<>();
        Deque<String> ops = new ArrayDeque<>();

        if (showTrace) {
            System.out.println("\n--- Step-by-step Trace (Algorithm B) ---");
            System.out.println("Token | Operator Stack | Operand Stack");
            System.out.println("-----");
        }

        for (String token : tokens) {
            comp++;
        }
    }
}
```

```

        if (Character.isDigit(token.charAt(0))) {
            values.push(Integer.parseInt(token));
            push++;
        } else if (token.equals("(")) {
            ops.push(token);
            push++;
        } else if (token.equals(")")) {
            while (!ops.isEmpty() && !ops.peek().equals("(")) {
                int[] c = applyTop(values, ops);
                push += c[0]; pop += c[1]; comp += c[2];
            }
            if (ops.isEmpty()) throw new ArithmeticException("Invalid parentheses.");
            ops.pop(); pop++;
        } else {
            while (!ops.isEmpty()
                && Tokenizer.precedence(ops.peek()) >= Tokenizer.precedence(token))
            {
                comp++;
                int[] c = applyTop(values, ops);
                push += c[0]; pop += c[1]; comp += c[2];
            }
            ops.push(token); push++;
        }

        if (showTrace) {
            printStackState(token, ops, values);
        }
    }

    while (!ops.isEmpty()) {

```



```

        int[] c = applyTop(values, ops);

        push += c[0]; pop += c[1]; comp += c[2];

        if (showTrace) {
            printStackState("End/Apply", ops, values);
        }
    }

    long end = System.nanoTime();

    return new EvaluationResult("N/A (Direct Infix)", values.pop(),
        end - start, push, pop, comp);
}

private static void printStackState(String currentToken, Deque<String> ops,
Deque<Integer> values) {
    Object[] opsArray = ops.toArray();
    Object[] valuesArray = values.toArray();

    StringBuilder opsStr = new StringBuilder("[");
    for (int i = opsArray.length - 1; i >= 0; i--) {
        opsStr.append(opsArray[i]).append(i > 0 ? ", " : "");
    }
    opsStr.append("]");

    StringBuilder valStr = new StringBuilder("[");
    for (int i = valuesArray.length - 1; i >= 0; i--) {
        valStr.append(valuesArray[i]).append(i > 0 ? ", " : "");
    }
    valStr.append("]");

    System.out.printf("%-5s | %-14s | %s\n", currentToken, opsStr.toString(),

```

```

valStr.toString());

    }

    private static int[] applyTop(Deque<Integer> values, Deque<String> ops) {

        String op = ops.pop();

        int b = values.pop();

        int a = values.pop();

        if (op.equals("/") && b == 0) throw new ArithmeticException("Division by zero.");

        int r;

        switch (op) {

            case "+": r = a + b; break;

            case "-": r = a - b; break;

            case "*": r = a * b; break;

            case "/": r = a / b; break;

            default: throw new IllegalArgumentException("Unsupported operator: " + op);

        }

        values.push(r);

        return new int[]{1, 3, 0};

    }

}

```

src/utls/ExpressionValidator.java

```
package group4.utls;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

public class ExpressionValidator {

    public static void validate(String expr) {

        if (expr == null || expr.trim().isEmpty()) {

            throw new IllegalArgumentException("Expression is empty.");

        }

        List<String> tokens = Tokenizer.tokenize(expr);

        if (tokens.isEmpty()) {

            throw new IllegalArgumentException("Expression contains no valid tokens.");

        }

        Deque<Character> stack = new ArrayDeque<>();

        for (char c : expr.toCharArray()) {

            if (c == '(') stack.push(c);

            else if (c == ')') {

                if (stack.isEmpty()) {

                    throw new IllegalArgumentException("Unbalanced parentheses: Extra ')'
found.");

                }

                stack.pop();

            }

        }

    }

}
```

```

    }

    if (!stack.isEmpty()) {
        throw new IllegalArgumentException("Unbalanced parentheses: Missing ')' at end.");
    }

    boolean expectOperand = true;
    for (String token : tokens) {
        if (Tokenizer.isOperator(token)) {
            if (expectOperand) {
                throw new IllegalArgumentException("Invalid operator sequence near '" +
token + "'");
            }
            expectOperand = true;
        } else if (token.equals("(")) {
            if (!expectOperand) {
                throw new IllegalArgumentException("Missing operator before '('");
            }
            expectOperand = true;
        } else if (token.equals(")")) {
            if (expectOperand) {
                throw new IllegalArgumentException("Invalid ')' placement");
            }
            expectOperand = false;
        } else {
            if (!expectOperand) {
                throw new IllegalArgumentException("Missing operator between operands");
            }
            expectOperand = false;
        }
    }
}

```

```
        if (expectOperand) {  
            throw new IllegalArgumentException("Expression ends with an operator or empty  
parentheses.");  
        }  
    }  
}
```

src/models/EvaluationResult.java

```
package group4.models;

public class EvaluationResult {

    private final String postfixExpression;

    private final int result;

    private final long executionTimeNs;

    private final int pushCount;

    private final int popCount;

    private final int comparisonCount;

    public EvaluationResult(String postfixExpression, int result, long executionTimeNs,
                           int pushCount, int popCount, int comparisonCount) {

        this.postfixExpression = postfixExpression;

        this.result = result;

        this.executionTimeNs = executionTimeNs;

        this.pushCount = pushCount;

        this.popCount = popCount;

        this.comparisonCount = comparisonCount;
    }

    public String getPostfixExpression() { return postfixExpression; }

    public int getResult() { return result; }

    public long getExecutionTimeNs() { return executionTimeNs; }

    public int getPushCount() { return pushCount; }

    public int getPopCount() { return popCount; }

    public int getComparisonCount() { return comparisonCount; }
```

```
public String getOpCount() {  
    return "Push=" + pushCount + ", Pop=" + popCount + ", Comp=" + comparisonCount;  
}  
}
```

src/algorithms/AlgorithmA.java

```
package group4.algorithms;

import group4.models.EvaluationResult;
import group4.utils.Tokenizer;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;

public class AlgorithmA {

    public static EvaluationResult evaluate(String expr) {

        long start = System.nanoTime();

        int push = 0, pop = 0, comp = 0;

        List<String> tokens = Tokenizer.tokenize(expr);

        List<String> postfix = new ArrayList<>();

        Deque<String> opStack = new ArrayDeque<>();

        for (String token : tokens) {

            comp++;

            if (Character.isDigit(token.charAt(0))) {

                postfix.add(token);

            } else if (token.equals("(")) {

                opStack.push(token); push++;

            } else if (token.equals(")")) {

                while (!opStack.isEmpty() && !opStack.peek().equals("(")) {

                    postfix.add(opStack.pop()); pop++;

                }

                opStack.pop(); pop++;

            }

        }

        return new EvaluationResult(start, postfix, comp);

    }

}
```



```

        }

        if (opStack.isEmpty()) throw new ArithmeticException("Invalid parentheses.");

        opStack.pop(); pop++;

    } else {

        while (!opStack.isEmpty()

                && Tokenizer.precedence(opStack.peek()) >=
Tokenizer.precedence(token)) {

            comp++;

            postfix.add(opStack.pop()); pop++;

        }

        opStack.push(token); push++;

    }

}

while (!opStack.isEmpty()) {

    String op = opStack.pop();

    if (op.equals("(")) throw new ArithmeticException("Invalid parentheses.");

    postfix.add(op); pop++;

}

Deque<Integer> values = new ArrayDeque<>();

for (String token : postfix) {

    if (Character.isDigit(token.charAt(0))) {

        values.push(Integer.parseInt(token)); push++;

    } else {

        if (values.size() < 2) throw new ArithmeticException("Invalid postfix
structure.");

        int b = values.pop(), a = values.pop(); pop += 2;

        if (token.equals("/") && b == 0) throw new ArithmeticException("Division by
zero.");

```

```

        values.push(apply(a, b, token)); push++;

    }

}

long end = System.nanoTime();

return new EvaluationResult(String.join(" ", postfix), values.pop(),

    end - start, push, pop, comp);

}

private static int apply(int a, int b, String op) {

    switch (op) {

        case "+": return a + b;

        case "-": return a - b;

        case "*": return a * b;

        case "/": return a / b;

        default: throw new IllegalArgumentException("Unsupported operator: " + op);

    }

}

}

```

src/algorithms/AlgorithmB.java

```
package group4.algorithms;

import group4.models.EvaluationResult;
import group4.utils.Tokenizer;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

public class AlgorithmB {

    public static EvaluationResult evaluate(String expr) {

        long start = System.nanoTime();

        int push = 0, pop = 0, comp = 0;

        List<String> tokens = Tokenizer.tokenize(expr);

        Deque<Integer> values = new ArrayDeque<>();

        Deque<String> ops = new ArrayDeque<>();

        for (String token : tokens) {

            comp++;

            if (Character.isDigit(token.charAt(0))) {

                values.push(Integer.parseInt(token)); push++;

            } else if (token.equals("(")) {

                ops.push(token); push++;

            } else if (token.equals(")")) {

                while (!ops.isEmpty() && !ops.peek().equals("(")) {

                    int[] c = applyTop(values, ops);

                    push += c[0]; pop += c[1]; comp += c[2];

                }

                ops.pop();

            }

        }

        return new EvaluationResult(comp, push, pop);
    }

    private static int[] applyTop(Deque<Integer> values, Deque<String> ops) {

        int[] c = new int[3];

        String op = ops.pop();

        if (op.equals("+")) {

            c[0] = values.pop() + values.pop();

        } else if (op.equals("-")) {

            c[0] = values.pop() - values.pop();

        } else if (op.equals("*")) {

            c[0] = values.pop() * values.pop();

        } else if (op.equals("/")) {

            c[0] = values.pop() / values.pop();

        }

        c[1] = values.size();
        c[2] = 1;

        return c;
    }
}
```

```

        }

        if (ops.isEmpty()) throw new ArithmeticException("Invalid parentheses.");

        ops.pop(); pop++;
    } else {
        while (!ops.isEmpty()
                && Tokenizer.precedence(ops.peek()) >= Tokenizer.precedence(token)) {
            comp++;

            int[] c = applyTop(values, ops);

            push += c[0]; pop += c[1]; comp += c[2];
        }

        ops.push(token); push++;
    }
}

while (!ops.isEmpty()) {
    int[] c = applyTop(values, ops);

    push += c[0]; pop += c[1]; comp += c[2];
}

long end = System.nanoTime();

return new EvaluationResult("N/A (Direct Infix)", values.pop(),
        end - start, push, pop, comp);
}

private static int[] applyTop(Deque<Integer> values, Deque<String> ops) {
    String op = ops.pop();

    int b = values.pop();

    int a = values.pop();

```

```
        if (op.equals("/") && b == 0) throw new ArithmeticException("Division by zero.");

        int r;

        switch (op) {

            case "+": r = a + b; break;

            case "-": r = a - b; break;

            case "*": r = a * b; break;

            case "/": r = a / b; break;

            default: throw new IllegalArgumentException("Unsupported operator: " + op);

        }

        values.push(r);

        return new int[]{1, 3, 0};

    }

}
```

src/ExpressionGenerator.java

```
package group4;

import java.util.Random;

public class ExpressionGenerator {

    private static final char[] OPS = {'+', '-', '*', '/'};

    public static String generate(int numberOfOperands, long seed) {

        Random random = new Random(seed);

        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < numberOfOperands; i++) {

            int value = 1 + random.nextInt(9);

            sb.append(value);

            if (i < numberOfOperands - 1) {

                sb.append(' ').append(OPS[random.nextInt(OPS.length)]).append(' ');

            }

        }

        return sb.toString();

    }

}
```

src/ExperimentRunner.java

```
package group4;

import group4.algorithms.AlgorithmA;
import group4.algorithms.AlgorithmB;
import group4.models.EvaluationResult;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class ExperimentRunner {

    private static final int[] SIZES = {100, 1000, 10000, 50000};

    private static final int ROUNDS = 5;

    public static void main(String[] args) throws IOException {

        new File("results").mkdirs();

        run("results/experiment_results.csv");

    }

    public static void run(String outputCsvPath) throws IOException {

        try (PrintWriter csv = new PrintWriter(new FileWriter(outputCsvPath))) {

            csv.println("n,algorithm,avg_time_ns,avg_push,avg_pop,avg_comparisons");

            for (int n : SIZES) {

                long[] ta = new long[ROUNDS], pa = new long[ROUNDS], qa = new long[ROUNDS],
                ca = new long[ROUNDS];

                long[] tb = new long[ROUNDS], pb = new long[ROUNDS], qb = new long[ROUNDS],
```

```

cb = new long[ROUNDS];

for (int r = 0; r < ROUNDS; r++) {

    String expr = ExpressionGenerator.generate(n, 1000L * n + r);

    EvaluationResult a = AlgorithmA.evaluate(expr);

    ta[r] = a.getExecutionTimeNs();
    pa[r] = a.getPushCount();
    qa[r] = a.getPopCount();
    ca[r] = a.getComparisonCount();

    EvaluationResult b = AlgorithmB.evaluate(expr);

    tb[r] = b.getExecutionTimeNs();
    pb[r] = b.getPushCount();
    qb[r] = b.getPopCount();
    cb[r] = b.getComparisonCount();

}

write(csv, n, "A", avg(ta), avg(pa), avg(qa), avg(ca));
write(csv, n, "B", avg(tb), avg(pb), avg(qb), avg(cb));

}

}

System.out.println("Saved results to: " + outputCsvPath);

}

private static void write(PrintWriter csv, int n, String alg,

    double t, double p, double q, double c) {

```



```
        csv.printf("%d,%s,%.0f,%.1f,%.1f,%.1f%n", n, alg, t, p, q, c);
    }

    private static double avg(long[] v) {
        long sum = 0;
        for (long x : v) sum += x;
        return (double) sum / v.length;
    }
}
```